

پروژه ۶: تحلیل نقصان‌های برخورد و طراحی معماری حافظه نهان قربانی^{۱۳۸}

در این پروژه، قصد داریم به یکی از چالش‌های مهم در طراحی حافظه نهان، یعنی افت عملکرد ناشی از نقصان‌های برخورد^{۱۳۹} بپردازیم. این مشکل به‌طور ویژه در حافظه‌های نهانی که با روش نگاشت مستقیم^{۱۴۰} یا انجمنی^{۱۴۱} با درجه پایین کار می‌کنند، رخ می‌دهد. این پروژه در شبیه‌ساز آکیتا پیاده‌سازی و تحلیل می‌شود.

شرح مسئله و ساختار کلی

معمولاً به دلیل محدودیت‌هایی که در ساختارهای نگاشت حافظه وجود دارد، ممکن است چندین آدرس پرکاربرد دقیقاً به یک خط از حافظه نهان اختصاص یابند. حتی اگر ظرفیت کلی حافظه نهان برای نگهداری داده‌ها کافی باشد، تداخل آدرس‌ها باعث می‌شود داده‌ها پیوسته از حافظه اخراج شده و پدیده‌ای به نام تلاطم^{۱۴۲} رخ دهد. این اتفاق، سامانه را مجبور می‌کند تا داده‌ها را بارها از سطوح پایین‌تر حافظه واکشی کند که جریمه زمانی و تأخیر قابل توجهی را به همراه دارد.

در ابتدا این رفتار ناکارآمد را در شبیه‌ساز مدل‌سازی کرده و شمارنده‌هایی برای ثبت نقصان‌های برخورد استفاده کنید. سپس با تغییر معماری و اضافه کردن یک پیمانه جدید به نام «حافظه نهان قربانی»، راه‌حلی برای کاهش ترافیک اضافی حافظه ارائه دهید. در نهایت باید نشان دهید که این راه‌حل چه تأثیری بر موازنه میان هزینه سخت‌افزاری و زمان اجرا دارد.

اهداف پروژه

- درک عملی پدیده‌هایی مانند اخراج مکرر داده‌ها، تلاطم حافظه و تشخیص تفاوت میان نقصان برخورد و نقصان ظرفیتی^{۱۴۳}،
- تحلیل تأثیر میزان انجمنی بودن حافظه بر نرخ اصابت و ترافیک حافظه،
- طراحی و پیاده‌سازی سخت‌افزاری یک پیمانه جدید برای حافظه نهان قربانی و مدیریت فرایند نسبتاً پیچیده جابه‌جایی دوطرفه^{۱۴۴}.

توضیح مفهومی عملکرد حافظه نهان قربانی

در معماری‌های پایه و مرسوم، وقتی یک بلاک جدید به دلیل نقصان عدم اصابت وارد حافظه نهان L1 می‌شود، بلاک قدیمی موجود در آن جایگاه بلافاصله اخراج^{۱۴۵} شده و به L2 فرستاده می‌شود. حال اگر پردازنده دوباره به همان بلاک اخراج‌شده نیاز پیدا کند، باید تأخیر سنگین خواندن دوباره آن از L2 را تحمل کند.

¹³⁸Victim Cache

¹³⁹Conflict Misses

¹⁴⁰Direct-Mapped

¹⁴¹Associative

¹⁴²Thrashing

¹⁴³Capacity Miss

¹⁴⁴Swap

¹⁴⁵Evict

برای حل این مشکل، یک بافر بسیار کوچک تمام‌انجمنی^{۱۴۶} بین L1 و L2 قرار می‌گیرد که به آن حافظه نهان قربانی می‌گوییم. روند کار به این صورت است که بلاک‌های اخراجی از L1 ابتدا وارد این بافر می‌شوند. اگر پردازنده داده‌ای را درخواست کند که در L1 نیست، اما داخل حافظه نهان قربانی وجود دارد، فرایندی به نام جابه‌جایی رخ می‌دهد؛ یعنی داده درخواستی سریعاً به L1 برمی‌گردد و هم‌زمان، بلاک اخراجی جدید از L1 جای آن را در حافظه نهان قربانی می‌گیرد. با این روش، از ارسال درخواست‌های اضافی و پرهزینه به L2 جلوگیری می‌شود.

مراحل پیاده‌سازی و نکات فنی

۱. پیاده‌سازی ساختار پایه

- طراحی یک برنامه محک^{۱۴۷} در شبیه‌ساز آکیتا با هدف ایجاد ترافیک متناوب. فاصله آدرس‌ها^{۱۴۸} در این برنامه باید مضربی از ظرفیت حافظه نهان باشد تا بتواند بیشترین میزان نقصان برخورد را شبیه‌سازی کند.
- اضافه کردن شمارنده‌های سخت‌افزاری برای ثبت تعداد دفعات اصابت/نقصان در L1 و تعداد درخواست‌های خواندن/نوشتن ارسالی به L2.

۲. اندازه‌گیری و مشاهده افت عملکرد

- اجرای برنامه‌های محک روی حافظه نهان نگاشت مستقیم، یعنی ساختار پایه.
- محاسبه میانگین زمان دسترسی به حافظه^{۱۴۹} در حالت پایه.
- نشان دادن میزان ترافیک هدررفته و تأخیر تحمیل‌شده به پردازنده در اثر پدیده تلاطم^{۱۵۰}.

۳. تحلیل معماری حافظه نهان

- بررسی ماشین حالت^{۱۵۱} ارتباطی بین L1 و L2 در معماری پایه،
- تحلیل مسیر رویدادهای خواندن و نوشتن و نحوه مدیریت پیام‌های مربوط به اخراج داده‌ها،
- بررسی چالش بن‌بست^{۱۵۲} در تبادل هم‌زمان پیام‌ها بین عناصر^{۱۵۳}.

۴. طراحی و پیاده‌سازی راه‌حل

- مسیریابی پیام‌ها^{۱۵۴}: تغییر منطق مسیریابی تا بلاک‌های اخراجی از L1 به‌جای رفتن به L2، مستقیماً به حافظه نهان قربانی هدایت شوند،

¹⁴⁶Fully Associative

¹⁴⁷Benchmark

¹⁴⁸Stride

¹⁴⁹Average Memory Access Time - AMAT

¹⁵⁰Thrashing

¹⁵¹State Machine

¹⁵²Deadlock

¹⁵³Components

¹⁵⁴Message Routing

- فرایند جابه‌جایی: مدیریت ناهمگام^{۱۵۵} رویدادها به‌گونه‌ای که در صورت اصابت در حافظه قربانی، بلاک مورد نیاز به‌طور ایمن به L1 برود و بلاک اخراجی جدید وارد پیمانه قربانی شود،
- سیاست جایگزینی: روشی مانند FIFO یا LRU برای مدیریت سرریز ظرفیت محدود حافظه نهان قربانی، برای مثال اندازه ۸ بلاک، به سمت L2 پیاده‌سازی شود.

۵. مقایسه قبل و بعد از اعمال حافظه نهان قربانی

- اجرای دوباره برنامه‌های محک پس از اعمال تغییرات،
- استخراج زمان کل اجرا^{۱۵۶} و محاسبه نرخ اصابت در حافظه نهان قربانی،
- مقایسه میزان کاهش ترافیک L2 و محاسبه فرمول ارتقایافته AMAT به شکل زیر:

$$AMAT = HitTime_{L1} + MissRate_{L1} \times (HitRate_{VC} \times Penalty_{VC} + MissRate_{VC} \times Penalty_{L2})$$

خروجی‌های مورد انتظار در گزارش و کدهای تحویلی

گزارش نهایی شما باید شامل موارد زیر باشد:

- رسم نمودار ماشین حالت و مسیریابی پیام‌ها در هر دو معماری پایه و دارای حافظه نهان قربانی،
- توضیحات دقیق درباره نحوه مدل‌سازی و مدیریت فرایند هم‌زمان جابه‌جایی بدون بروز بن‌بست در شبیه‌ساز،
- تحلیل ریاضی و فرمولی AMAT در دو حالت قبل و بعد از اضافه شدن پیمانه،
- شرح تغییرات اعمال‌شده در کد شبیه‌ساز و فایل‌های پیکربندی^{۱۵۷} شبکه حافظه‌های نهان،
- نمودارهای مقایسه‌ای که میزان کاهش ترافیک حافظه L2 را نشان دهند.

علاوه بر گزارش، نتایج آزمون شما نیز باید شامل این موارد باشد:

- کدهای پیاده‌سازی‌شده، یا عناصر نوشته‌شده، به زبان Go،
- ارائه مشاهدات عملی از نقصان‌های برخورد در حالت پایه با استفاده از اعداد دقیق به‌دست‌آمده از شمارنده‌ها،
- مقایسه کمی تأخیرهای سامانه و تأثیر آن‌ها بر زمان اجرای نهایی برنامه‌ها.

¹⁵⁵ Asynchronous

¹⁵⁶ Execution Time

¹⁵⁷ Configuration